Merge In Merge Sort

[1, 5, 7, 8]
[2, 3, 9, 10]

=> [1, 2, 3, 5, 7, 8, 9, 10]

[1, 2, 3, 5, 7, 8, 9, 10]

int i = 0;
int j = 0;

Merge 3 Sorted Arrays

A: [..]
B: [..]
C: [..]

int i = 0;
int j = 0;
Int k = 0;

Merge K Sorted Arrays:

K = 4
[
  [**4**, 8, 10, 12],
  [**1**, 1, 3, 5],
  [**23**, 25, 30, 100],
  [**4**, 9, 11, 15]
]

Output: [1, 1, 3, 4, 4, 5, 8, 9, 10, 11, 12, 15, 23, 25, 30, 100]

Brute Force:

[4, 8, 10, 12] + [1, 1, 3, 5]
[1, 1, 3, 4, 5, 8, 10, 12]

[23, 25, 30, 100] + [1, 1, 3, 4, 5, 8, 10, 12]:
[1, 1, 3, 4, 5, 8, 10, 12, 23, 25, 30, 100]

[4, 9, 11, 15] + [1, 1, 3, 4, 5, 8, 10, 12, 23, 25, 30, 100]:
[1, 1, 3, 4, 4, 5, 8, 9, 10, 11, 12, 15, 23, 25, 30, 100]

TC:
k + k
k + 2k
k + 3k
…
k + (k-1)k

(k-1)*k + [k + 2k + 3k + …. + (k-1)*k]

(k-1)*k + k[1 + 2 + 3 … (k - 1)]

(k-1)*k + k(k-1)k/2 = O(k^3)

AS: O(k^2)

Using K pointers:

[
  [**4**, 8, 10, 12],
  [1, **1**, 3, 5],
  [**23**, 25, 30, 100],
  [**4**, 9, 11, 15]
]
pointers: [0, 1, 0, 0]

result: [1, …]

TC: O(k * k^2) = O(k^3)
AS: O(k)

<u>Using K Pointers Stored in a Min-Heap:</u>

[
  [**4**, 8, 10, 12],
  [1, **2**, 3, 5],
  [**23**, 25, 30, 100],
  [**4**, 9, 11, 15]
]

In the PQ, we will store: (value, (i, j))

pointersPQ: [4(0,0), 2(1,1), 23(2,0), 4(3,0)]

Result: [1, …]

TC: O(k^2*log(k))
AS: O(k)

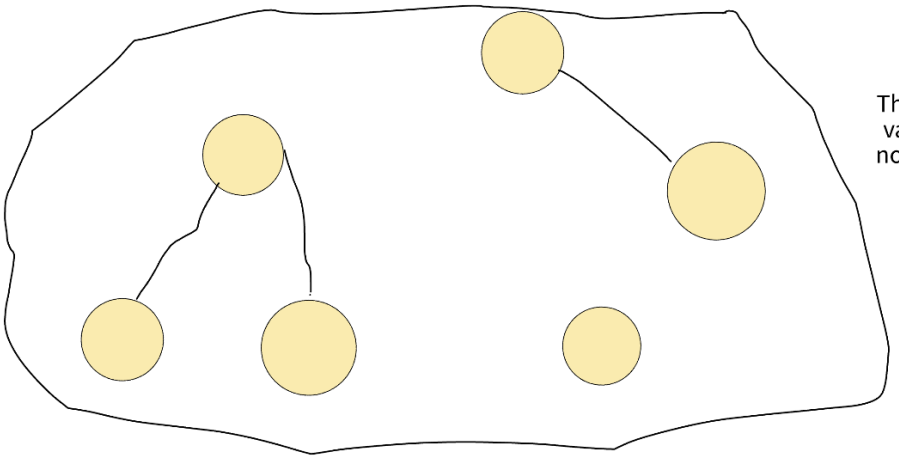
[
  [4, 8, **10**, 12],
  [1, 2, 3, 5],
  [**23**, 25, 30, 100],
  [4, 9, **11**, 15]
]

PQ:
{
  10 (0,2)
  23 (2,0)

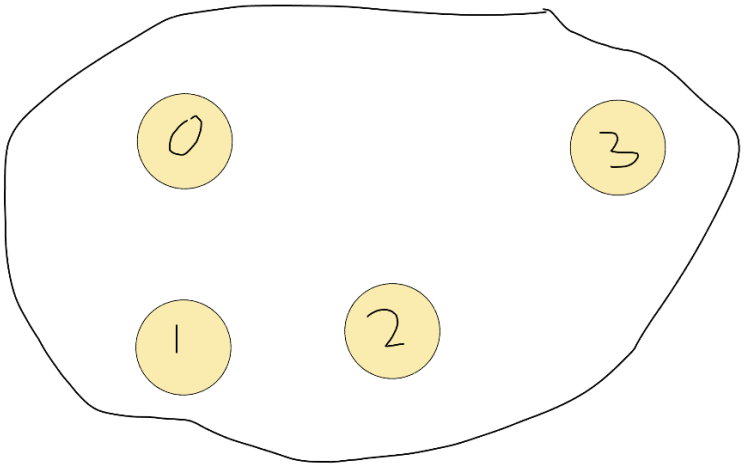
  11 (3,2)
}

result: [1, 2, 3, 4, 4, 5, 8, 9, …]

—----------

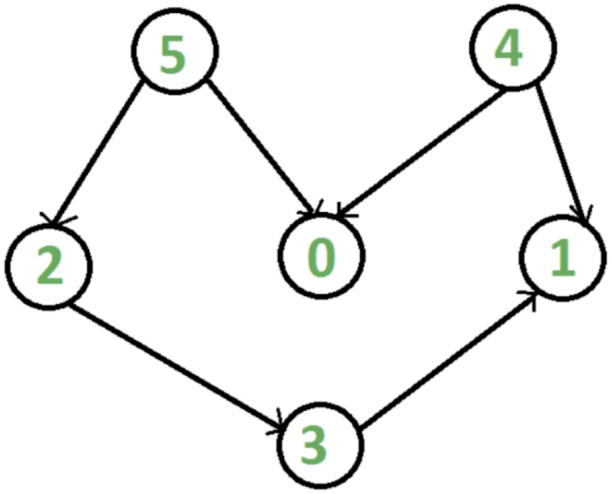


This as a whole is a valid graph with 6 nodes and 3 edges.



This as a whole is a valid graph with 4 nodes and 0 edges.

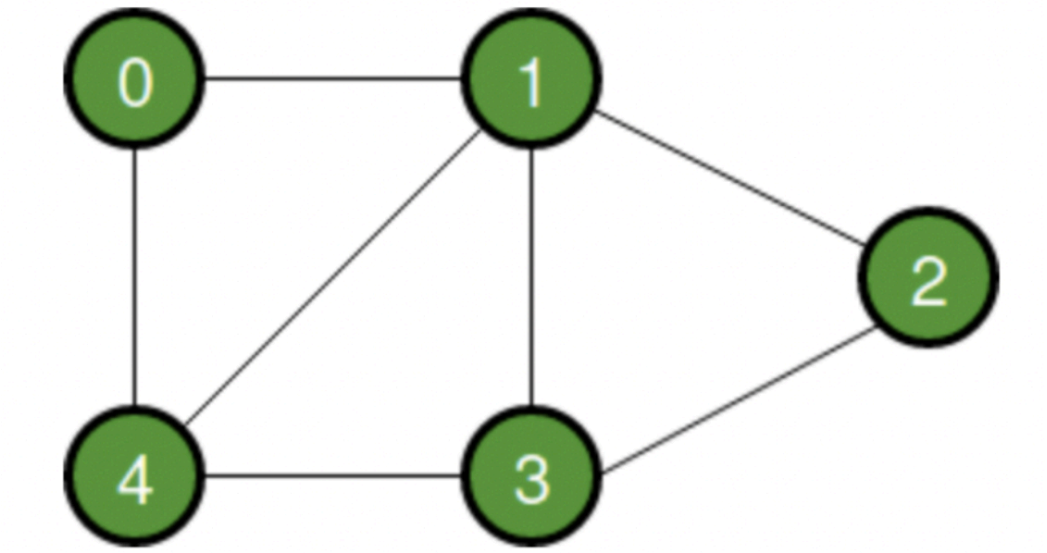
Directed Graphs:



Edge(u,v) -> means there is an edge from u to v, but not necessarily from v to u.
Edge(5, 2) != Edge(2, 5)

For eg: Social networks like Instagram (one-way connections of followings)

Undirected Graph:



Edge(u, v) -> can be used to go from u to v as well as from v to u.
Edge(0, 4) == Edge(4, 0)

For eg -> Social networks like Linkedin.

Representation:

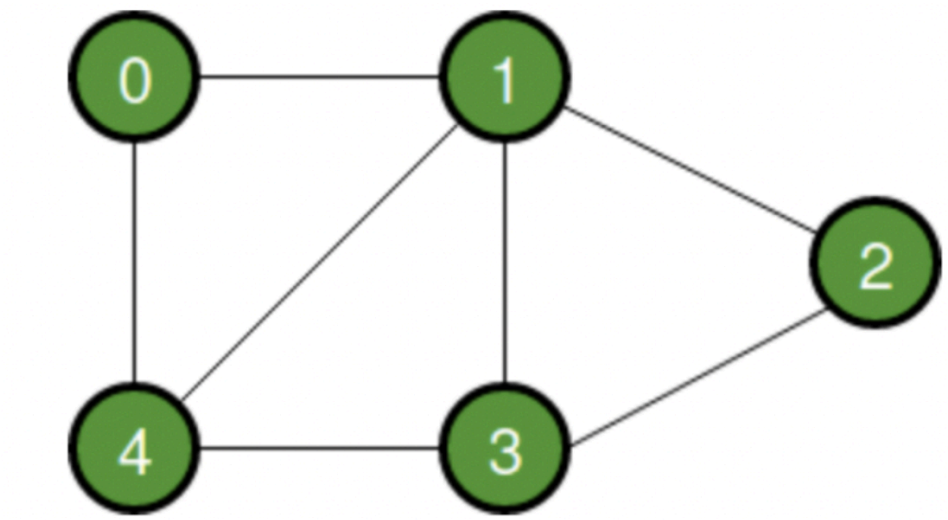
```
Class GraphNode {
        Int value;
```

```
        vector<GraphNode*> edges;
}


Class Person {
        Int id;
        String name;
        Int age;
        …
}
```

ReverseMapping: {int Id -> Person it belongs to}

Adjacency Matrix:



Adj[u][v] = 1 -> if there is an edge from u to v
Adj[u][v] = 0 -> if there is no edge from u to v

V = 5

	0	**1**	**2**	**3**	**4**
0	0	1	0	0	1

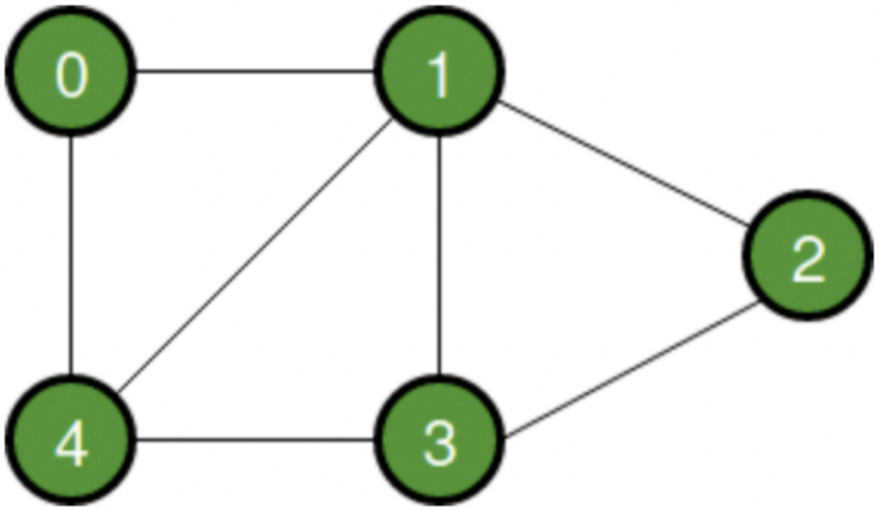
1	1	0	1	1	1
2	0	1	0	1	0
3	0	1	1	0	1
4	1	1	0	1	0

For weighted graphs:
Adj[u][v] = w -> if there is an edge from u to v with weight w.
Adj[u][v] = 0 -> if there is no edge from u to v

Space Inefficient!!
Suppose there are 1000 nodes and only 3 edges overall in the graph.
In that case .. the matrix will be of size 1000 x 1000
And only 3 values (out ot 1M) in the matrix will be equal to 1
And rest all the values will be equal to 0

Hence, we end up with a very sparse matrix!! -> which is very space inefficient.

Adjacency List

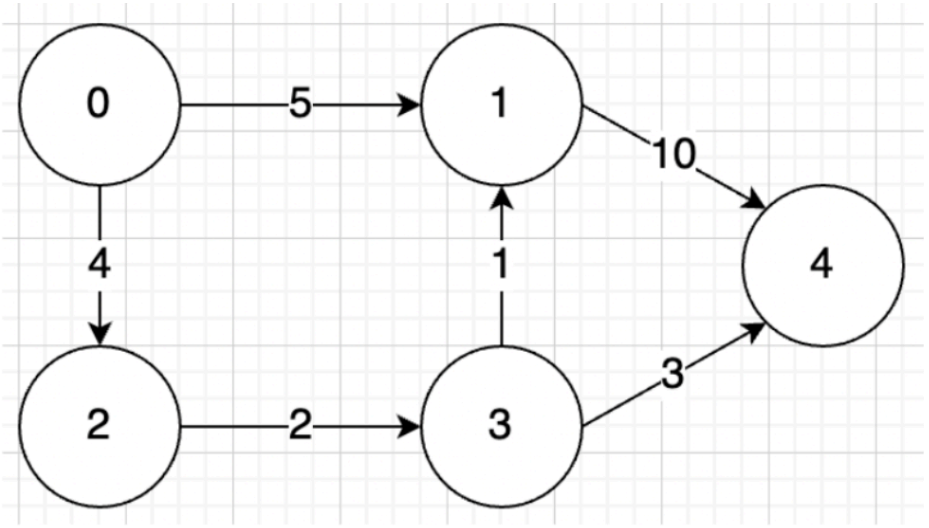


V = 5
[
  [1, 4]
  [0, 4, 3, 2]
  [1, 3]
  [1, 2, 4]
  [3, 0, 1]
]

For edge(u, v) : adj[u] contains v.

For weighted graphs:
For edge(u, v, w) : adj[u] contains pair (v, w)

Eg:



V = 5
[
  [(2, 4), (1, 5)]
  [(4, 10)]
  [(3, 2)]
  [(1, 1), (4, 3)]
  [ ]
]

Space: O(V + E)